

Set 5 - Encapsulation, Abstraction and Interfaces

Q1.

Create a Java class called Playlist with private fields: name (String), owner (String), and totalSongs (int). Add public getter and setter methods for each field, and write a main method to set and print all values.

Answer:

```
public class Playlist {
    private String name;
    private String owner;
    private int totalSongs;

    // Getters
    public String getName() { return name; }
    public String getOwner() { return owner; }
    public int getTotalSongs() { return totalSongs; }

    // Setters
    public void setName(String name) { this.name = name; }
    public void setOwner(String owner) { this.owner = owner; }
    public void setTotalSongs(int totalSongs) { this.totalSongs = totalSongs; }

    public static void main(String[] args) {
        Playlist playlist = new Playlist();
        playlist.setName("Weekend Vibes");
        playlist.setOwner("Rohan");
        playlist.setTotalSongs(25);

        System.out.println("Playlist Name: " + playlist.getName());
        System.out.println("Owner: " + playlist.getOwner());
        System.out.println("Total Songs: " + playlist.getTotalSongs());
    }
}

/* Sample Output:
Playlist Name: Weekend Vibes
Owner: Rohan
Total Songs: 25
*/
```

Q2.

Build a Flipkart-style Product class with fields: productName (String), price (double), and discount (double). Make all fields private, provide public getters/setters, and add a method getFinalPrice() that returns price after applying discount.

Answer:

```
public class Product {
    private String productName;
    private double price;
    private double discount;    // percentage, e.g. 10 means 10%

    public String getProductName() { return productName; }
    public void setProductName(String productName) { this.productName = productName; }
    public double getPrice() { return price; }
```

```

public void setPrice(double price) { this.price = price; }

public double getDiscount() { return discount; }
public void setDiscount(double discount) { this.discount = discount; }

public double getFinalPrice() {
    return price - (price * discount / 100);
}

public static void main(String[] args) {
    Product product = new Product();
    product.setProductName("Wireless Earbuds");
    product.setPrice(2500);
    product.setDiscount(20);

    System.out.println("Product: " + product.getProductName());
    System.out.println("Original Price: Rs. " + product.getPrice());
    System.out.println("Final Price after discount: Rs. " + product.getFinalPrice());
}

/* Sample Output:
Product: Wireless Earbuds
Original Price: Rs. 2500.0
Final Price after discount: Rs. 2000.0
*/

```

Q3.

Create an abstract class called `UPITransaction` with an abstract method `processTransaction(double amount)`. Then, make two subclasses: `PaytmTransaction` and `PhonePeTransaction`, each providing their own implementation of `processTransaction` that prints which app is processing the amount.

Answer:

```

abstract class UPITransaction {
    abstract void processTransaction(double amount);
}

class PaytmTransaction extends UPITransaction {
    @Override
    void processTransaction(double amount) {
        System.out.println("Paytm is processing a transaction of Rs. " + amount);
    }
}

class PhonePeTransaction extends UPITransaction {
    @Override
    void processTransaction(double amount) {
        System.out.println("PhonePe is processing a transaction of Rs. " + amount);
    }
}

public class UPIDemo {
    public static void main(String[] args) {
        UPITransaction t1 = new PaytmTransaction();
        UPITransaction t2 = new PhonePeTransaction();

        t1.processTransaction(500.0);
        t2.processTransaction(1200.0);
    }
}

```

```

/* Sample Output:
Paytm is processing a transaction of Rs. 500.0
PhonePe is processing a transaction of Rs. 1200.0
*/

```

Q4.

Write a Java interface called Searchable with a method search(String keyword). Implement this interface in a MovieLibrary class that stores an array of movie names, and returns all movies containing the keyword in search().

Answer:

```

interface Searchable {
    void search(String keyword);
}

class MovieLibrary implements Searchable {
    private String[] movies = {
        "The Dark Knight", "Inception", "Interstellar",
        "The Dark Tower", "Dark Waters", "Avengers Endgame"
    };

    @Override
    public void search(String keyword) {
        System.out.println("Search results for \"" + keyword + "\"");
        boolean found = false;
        for (String movie : movies) {
            if (movie.toLowerCase().contains(keyword.toLowerCase())) {
                System.out.println("- " + movie);
                found = true;
            }
        }
        if (!found) {
            System.out.println("No movies found.");
        }
    }

    public static void main(String[] args) {
        MovieLibrary library = new MovieLibrary();
        library.search("dark");
    }
}

/* Sample Output:
Search results for "dark":
- The Dark Knight
- The Dark Tower
- Dark Waters
*/

```

Q5.

Explain in your own words how abstraction in Java (using abstract classes or interfaces) acts like an API contract between the service layer and database layer in a Zomato-style food ordering app. Give one example for each (abstract class and interface) showing what methods would be exposed.

Answer:

Abstraction lets one layer of an application depend only on 'what' another layer does, not 'how' it does it - exactly like an API contract. The service layer that handles business logic (calculating totals, applying offers) doesn't need to know whether the database layer is MySQL, MongoDB, or an in-memory cache; it only needs to know the method names it can call and what they return. This keeps the layers loosely coupled, so the database layer can be swapped or upgraded without touching the service layer's code.

Abstract class example - OrderRepository (partly implemented, shared logic lives here):
`abstract class OrderRepository { abstract Order getOrderById(String orderId); abstract void saveOrder(Order order); }`

Interface example - PaymentGateway (a pure contract, since payment providers differ completely):
`interface PaymentGateway { boolean processPayment(double amount); void refund(String transactionId); }`

The service layer only calls `getOrderById()`, `saveOrder()`, `processPayment()` and `refund()` - it never touches SQL queries or a specific payment provider's SDK directly. That separation is the essence of abstraction acting as a contract.